

Software Requirements Specification

API Monitoring SaaS Platform

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for the API Monitoring Software as a Service (SaaS) platform. This platform enables companies to register their APIs, assign them to employee accounts, monitor real-time uptime, response times, and key health statuses, and receive instant alerts when anomalies occur.

1.2 Scope

The SaaS application consists of a full-stack web interface with role-based access control, real-time background monitoring engines, and an analytics dashboard. It monitors APIs via HTTP checks, validates third-party API keys (e.g., OpenAI, Stripe), tracks slow load degradation, caches analytics, and sends email alerts upon failures.

2. Overall Description

2.1 Product Features

1. Multi-tenant Role-Based Access Control (RBAC): Company Admins can invite employees, manage subscriptions, and view all system endpoints. Employees can manage and view only their assigned endpoints.
2. API Endpoint Monitoring: Real-time polling to verify status codes, response times, and API key validities.
3. Third-Party API Key Validation: Integrations specifically detecting INVALID, LIMITED, and RATE_LIMITED HTTP signatures for external providers.
4. Performance Degradation Detection: Algorithmic identification of progressively slowing response times under load before a total timeout occurs.
5. Real-time Analytics Dashboard: Data visualization charts for Uptime, Average Response Times, and Error Rates.
6. Alert Notification System: SMTP email alerts dispatched to API owners when APIs go down or keys become invalid.
7. Background Cron Execution: Scheduled tasks running at specific intervals (e.g., every 60s) to perform the endpoint checks without blocking the main web server.

8. Automated Data Purging: Regular garbage collection of expired email verification OTPs and old endpoint monitoring logs to preserve database efficiency.

2.2.2 Typical Data Flow

1. User Input: Company admins or Employees add an API URL and configuration via the frontend dashboard.
2. Endpoint Validation: The backend validates the URL and API key.
3. Database Storage: The Flask backend stores the endpoint details with correct ownership tracking (`user_id` and `created_by`) in PostgreSQL.
4. Monitoring Cycle: The background engine fetches active endpoints and pings them at defined intervals.
5. Logging: Results (status code, latency, API key status) are logged into the `monitoring_logs` table.
6. Notification: If failure criteria are met (consecutive failures or rate limits), the Alert Service triggers an SMTP email to the employee and company admin.
7. Visualization: The dashboard fetches aggregated metrics via the REST API to render Chart.js graphs.

2.2.3 User Classes and Characteristics

- Company Administrators: Users who own the primary account, have full visibility over all endpoints created by the company and its employees.
- Employees: Sub-accounts invited by the Company Admin. They can add, manage, and monitor only their own specific API endpoints.

2.2.4 Operating Environment

- Server Side: Python 3.10+, Flask, Waitress/Gunicorn, PostgreSQL (Supabase).
- Client Side: Any modern web browser (Chrome, Firefox, Safari, Edge).
- Hosting: Vercel (Web Server), Background Engine Threading.

2.2.5 Design and Implementation Constraints / Assumptions

- The system depends on the availability of the PostgreSQL API.
- Monitoring frequency is fixed at 60 seconds.
- SMTP delivery depends on the provider (e.g., Gmail) and correct App Password configuration.
- Serverless scaling on Vercel requires custom DBWrappers to prevent connection pool starvation.

2.2.6 How Dynamic Website Works

The system uses a Full-Stack Decoupled Architecture:

- The Frontend (HTML/CSS/JS) is a Single Page Application (SPA) style setup that communicates with the backend via fetch requests.
- The Backend (Flask) acts as a REST API server, handling authentication, data aggregation, and database

logic.

- The Background Engine runs asynchronously to ensure monitoring continues even without active user sessions.

2.2.7 Assumptions and Dependencies

- Assumed Browser: HTML5 and ES6 compatibility.
- Internet Discovery: Users must have a stable internet connection to access the dashboard.
- Email Service: Assumes the user provides a valid email address for alerts.

2.3 Functional Requirements

2.3.1 Core Functionality Description

The system provides a seamless experience for monitoring APIs across a company structure. Users can define endpoints (URL, Method, Interval, API keys), and the system ensures they are reachable and valid. The dashboard provides a "NOC-style" view of all monitored services, isolating data so employees only see their scope, while companies see aggregate overviews.

2.3.2 External Interface Requirements

2.3.2.1 User Interface

- Landing Page: Modern centered glassmorphic login/registration card.
- Dashboard: Sidebar navigation with dynamic content area, health status cards, and responsive Chart.js charts.
- Modals: For adding/editing endpoints and inviting employees.

2.3.2.2 Hardware Interfaces

No specialized hardware required. Standard Server/PC with NIC.

2.3.2.3 Software Interfaces

- Web Browser: Chrome 90+, Firefox 88+.
- Database: Supabase (PostgreSQL).
- Backend Framework: Flask (Python).
- Libraries: Chart.js for visualization, APScheduler for tasks.

2.3.2.4 Communication Interfaces

- HTTP/HTTPS: For client-server communication and API pings.
- SMTP: For sending critical failure alerts.

2.3.3 Detailed Functional Requirements

2.3.3.1 Web Pages Identified

- login.html: Entry point for all users.
- admin.html: The main control center showing global stats, charts, and endpoint controls.

Non-Functional Requirements

Security:

- Password hashing using bcrypt.
- Authorization via JWT in headers.
- Multi-Tenant Isolation using company_id and created_by attributes.
- API key masking in all responses (only first 4 and last 4 characters shown).
- Secure connection pool management.

Performance:

- API health check responses logged in less than 500ms.
- Dashboard stats aggregation returned in under 200ms using caching.

Scalability:

- PostgreSQL handles concurrent database connections reliably.
- Flask backend is stateless for horizontal serverless scaling.

Availability:

- Background monitoring engine ensures 24/7 pings.

Usability:

- Clean, premium design with intuitive navigation.

2.4 Advantages of the System

- Real-time Awareness: Users know immediately when their API is down or degrading.
- Data-Driven Insights: Uptime and latency charts help identify performance regressions.
 - Corporate Hierarchy Control: Perfect data isolation between company admins and employee sub-accounts.
- Secure by Design: Implementation of strict RBAC ensures user data privacy.

2.5 Future Enhancements

- Multi-region Pings: Monitor APIs from different global locations.
- Slack/Discord Webhooks: Notification support for dev teams.

- AI Diagnostics: Automatically analyze root causes of failures.
- Mobile Application: Native iOS/Android app for on-the-go alerts.